

Lab Manual for Computer Organization and Assembly Language

Lab-1

Introduction to Assembly Language

Lab Deliverables (submission guideline)

Create a **folder** named after your **enrollment number** and add the following files:

1. Create a **folder** named **YourRollNumber_Lab1** containing:
 - **.asm** files of the three practice tasks.
 - Add **full screen** screenshots of the code and output in this word file.
2. Zip the Folder and upload to LMS.

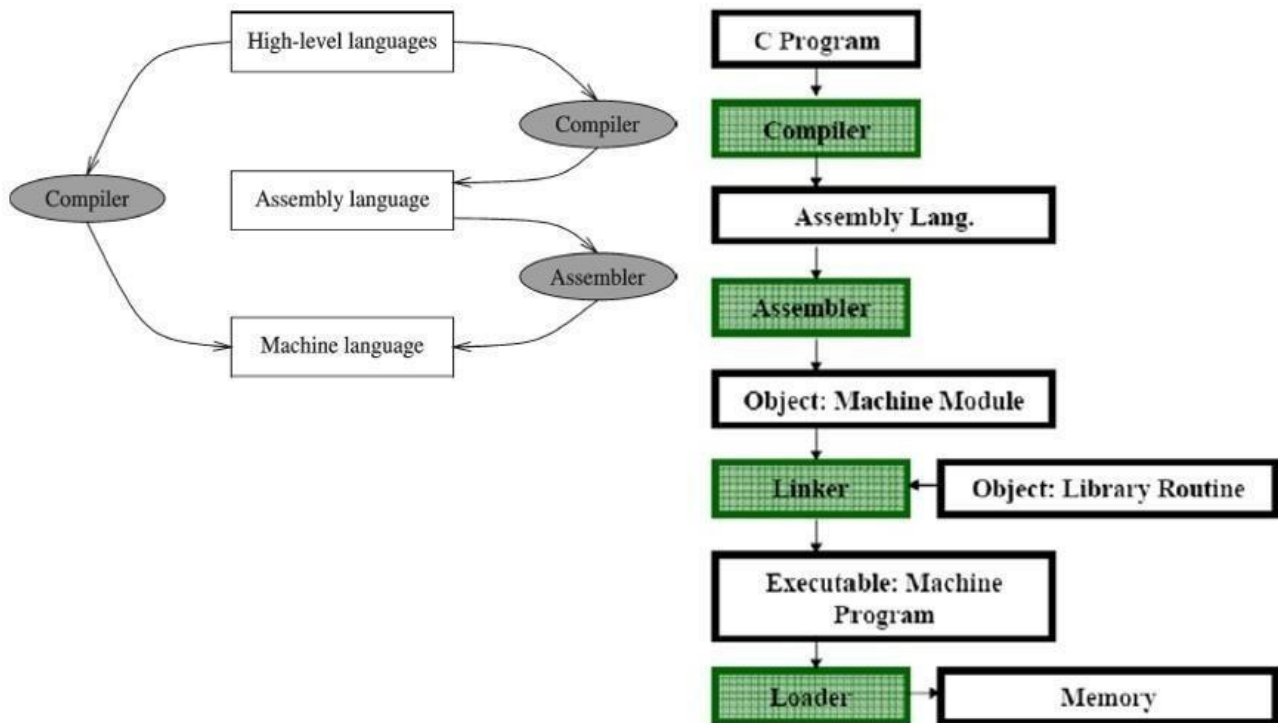
Academic Integrity Guidelines

- Lab work must be done individually.
- Copying code, screenshots, or files from others or online sources is strictly prohibited.
- Sharing solutions with classmates is considered cheating.
- Submissions will be checked for similarity in code, logic, and outputs.
- Plagiarism will result in zero marks and may result in F grade.
- Students are responsible for ensuring their submission is their own work.

1. Introduction

Machine language (computer's native language) is a system of impartible instructions executed directly by a computer's central processing unit (CPU).

- Instructions consist of binary code: 1s and 0s



The difference between compiling and interpreting is as follows. Compiling translates the high-level code into a target language code as a single unit. Interpreting translates the individual steps in a high-level program one at a time rather than the whole program as a single unit. Each step is executed immediately after it is translated.

C, C++ code is executed faster than Java code, because they transferred to assembly language before machine language.

2. Objective

- To get basic understanding of Assembly Language.
- To write simple assembly program and learn how to debug and run it
- To get an understanding of identifying basic errors.
- To understand the emu8086 emulator and their purpose.
- To be able to write basic assembly codes.

3. Concept Map

This section provides you with the overview of the concepts that will be discussed and implemented in this lab.

3.1. Assembly Language

Assembly Language is a programming language that is very similar to machine language, but uses symbols instead of binary numbers. It is converted by the assembler (e.g., NASM, TASM and MASM) into executable machine-language programs. Assembly language is machine-dependent; an assembly program can only be executed on a particular machine.

3.1.1. Introduction to Assembly Language Tools

Software tools are used for editing, assembling, linking, and debugging assembly language programming. You will need an assembler, a linker, a debugger, and an editor.

3.1.1.1. Assembler

An assembler is a program that converts source-code programs written in assembly language into object files in machine language. Popular assemblers have emerged over the years for the Intel family of processors. These include MASM (Macro Assembler from Microsoft), TASM (Turbo Assembler from Borland), NASM (Netwide Assembler for both Windows and Linux), and GNU assembler distributed by the free software foundation. We will use MASM 6.15 and TASM.

- Masm.exe creates an .obj file from an .asm file.

3.1.1.2. Linker

A linker is a program that combines your program's object file created by the assembler with other object files and link libraries, and produces a single executable program. You need a linker utility to produce executable files.

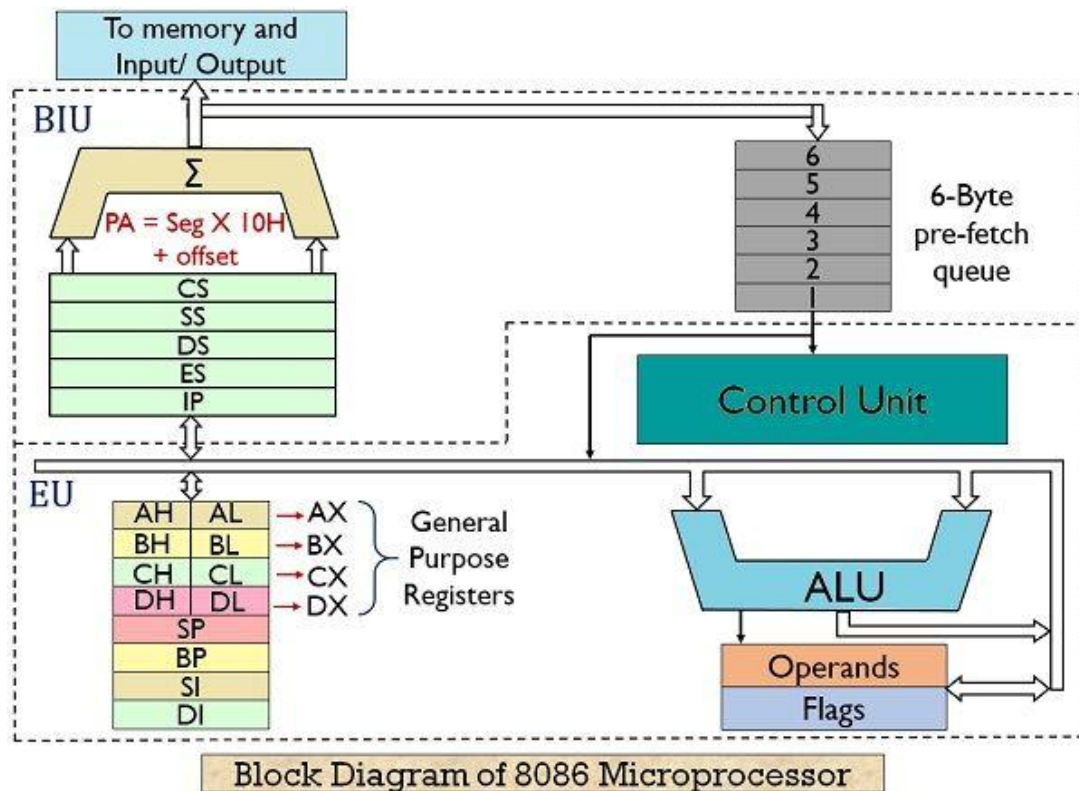
- Link.exe creates an .exe file from an .obj file.

3.1.1.3. Debugger

A debugger is a program that allows you to trace the execution of a program and examine the content of registers and memory. For 16-bit programs, MASM supplies a 16-bit debugger named CodeView. CodeView can be used to debug only 16-bit programs and is already provided with the MASM 6.15 distribution.

3.1.1.4. Editor

You need a text editor to create assembly language source files. MASM6.15 has its own editor or you can use for example Notepad++. In this course, we use microprocessor 8086 emulator for assembly language programs. The downloading link is given below:



Bus Interface Unit (BIU)

Functions

- Fetches instructions or data from memory or I/O
- Writes data to memory or I/O ports

Functional Components

1. Instruction Pointer (IP)

- 16 bit register
- Holds the offset address of the **next instruction** to be executed

2. Segment Registers

- Addressable memory space is **1 MB**, divided into four logical segments
- Each segment size is **64 KB**

Segment	Register	Purpose
Code	CS	Stores program instructions
Data	DS	Stores program data
Stack	SS	Manages stack operations
Extra	ES	Used for string operations

3. Instruction Queue

- 6 byte FIFO queue
- High speed register
- Stores prefetched instructions to improve execution speed

Execution Unit (EU)

Functions

- Fetches instructions from the BIU instruction queue
- Decodes instructions
- Executes arithmetic and logical operations using ALU

Functional Components

1. General Purpose Registers (GPR)

- 8086 has **four 16 bit registers**
- Each can be divided into **two 8 bit registers**

Register	16 bit	8 bit Parts	Function
Accumulator	AX	AH AL	Arithmetic operations, multiplication, division, string operations
Base	BX	BH BL	Offset address in indirect addressing
Count	CX	CH CL	Loop, shift, rotate, and string operation counter
Data	DX	DH DL	Works with AX for 32 bit multiplication and division

2. Pointer and Index Registers

Used for stack operations, addressing, and string manipulation.

- Stack Pointer (SP)
- Base Pointer (BP)
- Source Index (SI)
- Destination Index (DI)

3. Arithmetic Logic Unit (ALU)

- 16 bit ALU
- Performs both **8 bit and 16 bit** arithmetic and logical operations

4. Flag Register

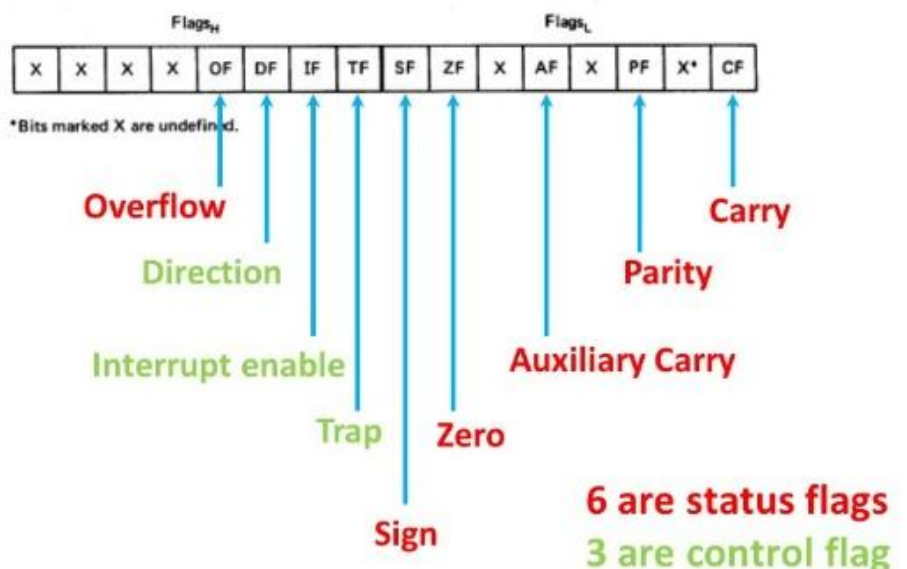
- Contains **9 active flags** (7 flags are unused)

Status Flags (6)

- Carry Flag (CF)
- Parity Flag (PF)
- Auxiliary Carry Flag (AF)
- Zero Flag (ZF)
- Sign Flag (SF)
- Overflow Flag (OF)

Control Flags (3)

- Trap Flag (TF)
- Interrupt Flag (IF)
- Direction Flag (DF)



Physical Address Formation

Physical Address = Segment Address × 10H + Offset Address

In 8086, memory is accessed using **segmented addressing**.

- The **segment address** is shifted left by **one hexadecimal digit** by multiplying it with **10H**
- The **offset address** is then added to locate the exact memory location

So,

Physical Address = Segment Address × 10H + Offset Address

This allows the 16 bit processor to access a **1 MB memory space**.

4. Sample Program

```
.model small

.data
msg db "Welcome to 1st Assembly Language Lab", "$"

.code
main:
    mov ax, @data
    mov ds, ax
    mov dx, offset msg

    mov ah, 9
    int 21h

    mov ah, 4ch
    int 21h
end main
```

4.1 Analysis of code:

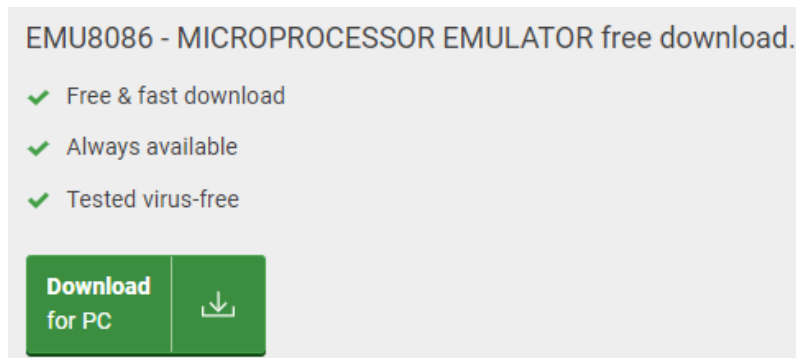
Instruction	Description
.MODEL SMALL	is a directive to tell the assembler to use one data segment and one code segment. All data fits in one 64K segment, all code fits in one 64K segment. Maximum program size is 128K.
.DATA	is a directive to put in data segment.
.CODE	is a directive to put in code segment.
@Data	is a default address of data segment to put it in ax register.
mov ax,@data mov ds,ax	As note we can't put data in ds register directly. So we use intermediate register (ax) as in the mov ds, ax
mov ah, 9	Put the service 9 in ah.
int 21h	(Interrupt 21 hexa), it has many services like 9,8,2, and each one as special work.
mov ah,4ch int 21h	The two statements to terminate the execution of the program.
end main	is a directive to indicate the end of the file

5. Tool Setup and procedure

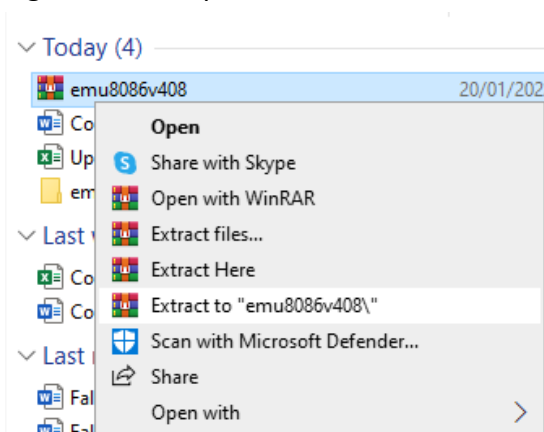
In this section, you will study how to setup and MASM Assembler

5.1. Tools

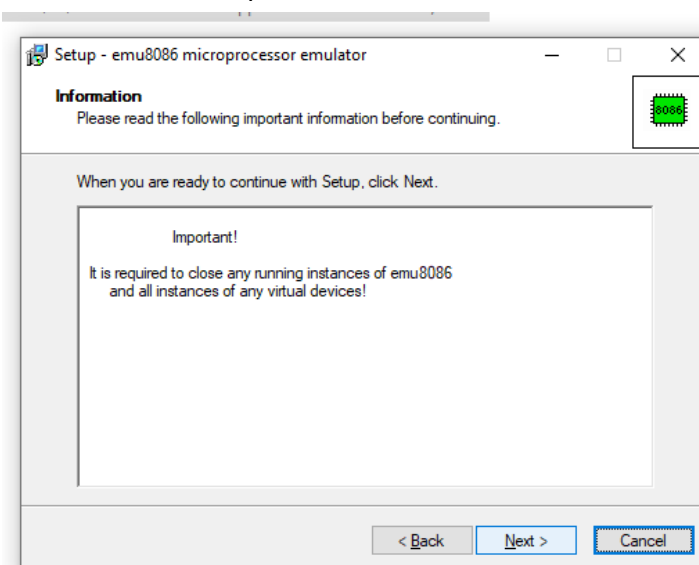
- Download emu8086 from
<https://emu8086-microprocessor-emulator.en.softonic.com/>

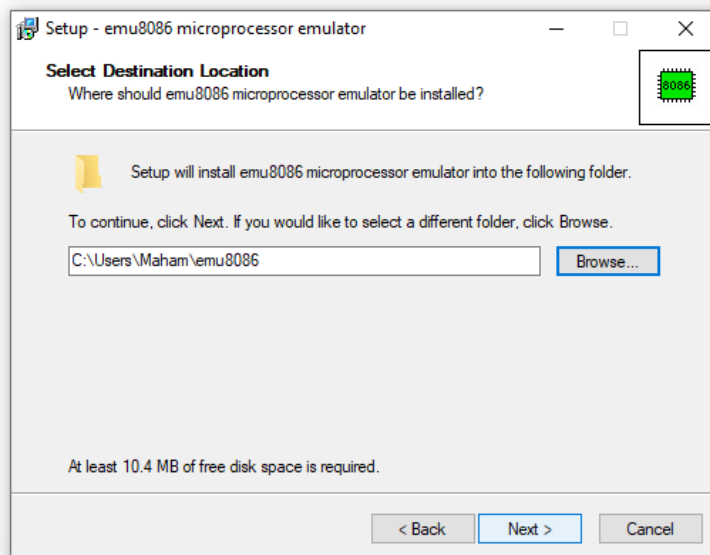


- Right-click on zip file and extract

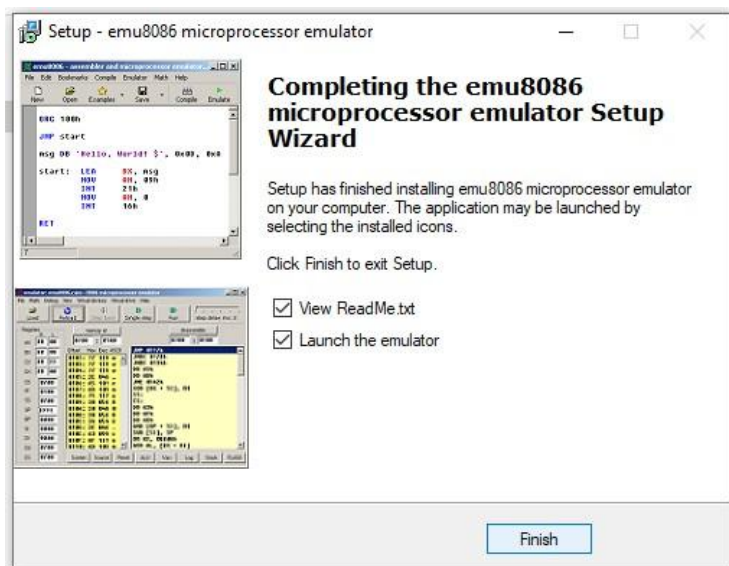


- Run the emu "Setup" from extracted folder

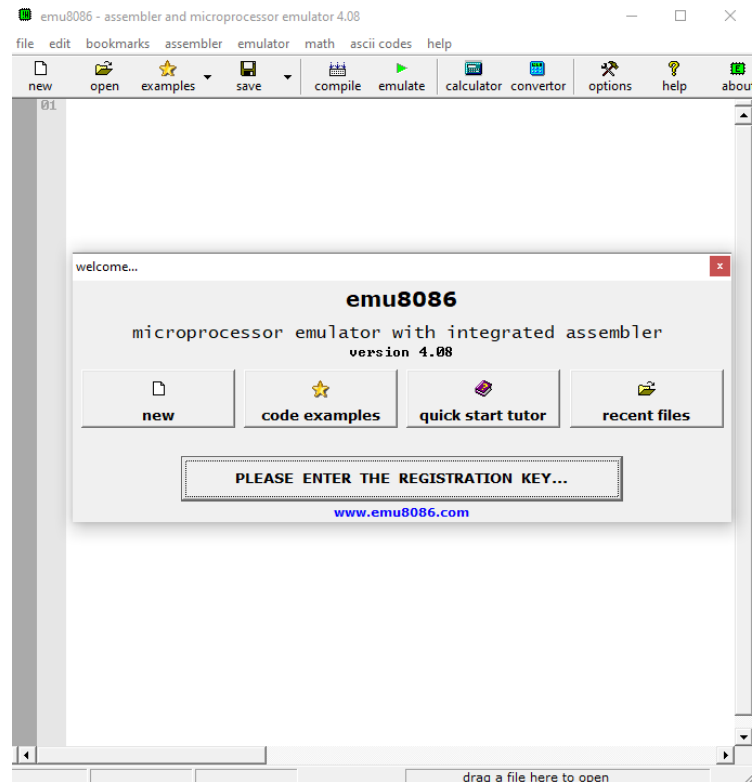




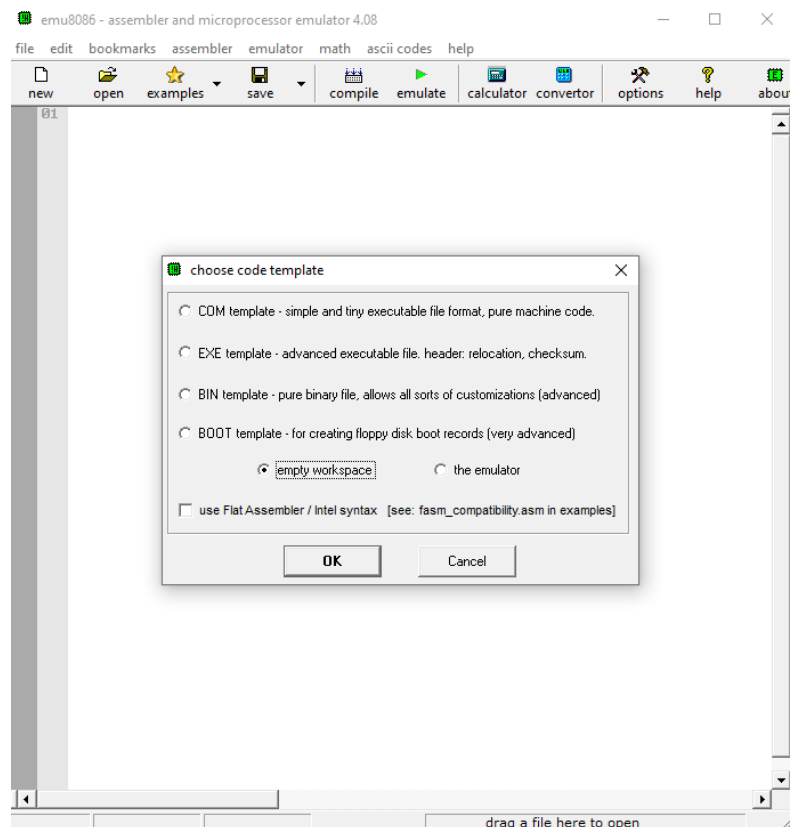
- Click “Next”

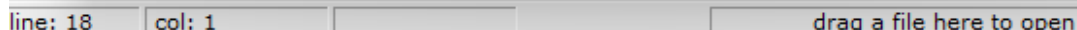


- Run emu8086



- Create new project with “empty workspace”





6. Practice Tasks

This section will provide more practice exercises which you need to finish during the lab. You need to finish the tasks in the required time.

6.1. Practice Task 1

[Expected time = 10mins]

Print your university card's detail on console in the proper format.

6.2. Practice Task 2

[Expected time = 10mins]

Write the following message on the console using (0Ah = line

```
feed): This
      is
      my
      "Name"
```

6.3. Practice Task 3

[Expected time = 10mins]

Print the following pattern:

```
*
**
***
****
*****
```

7. Out comes

After completing this lab, student will be able to setup emu8086. He/ She will also be able to compile and run basic Assembly programs.

8. Evaluation Task (Unseen)

[Expected time = 30mins for tasks]

Unseen task will be given depending upon the progress of the class.

9. Evaluation criteria

The evaluation criteria for this lab will be based on the completion of the tasks. Each task is assigned the marks percentage which will be evaluated by the instructor in the lab whether the student has finished the complete/partial task(s).

References:

Sample code **text**:

```
.model small      ; Selects SMALL memory model
                  ; One code segment and one data segment (max 64KB each)

.data            ; Start of data segment
msg db "Welcome to 1st Assembly Language Lab", "$"
               ; msg is a label (variable name)
               ; db means "Define Byte"
               ; Each character of the string is stored as 1 byte
               ; '$' is a string terminator required by
               ; DOS interrupt 21h, function 09h

.code            ; Start of code segment
main:            ; Program entry point label

mov ax, @data    ; @data gives the address of the data segment
                  ; Load that address into AX

mov ds, ax        ; Initialize DS register
                  ; DS must point to data segment before using variables

mov dx, offset msg ; OFFSET gives the address of msg inside data segment
                  ; DX holds the address of the string to be printed

mov ah, 9         ; AH = 09h selects DOS function
                  ; Function 09h displays a '$'-terminated string

int 21h           ; Call DOS interrupt
                  ; Prints the message on screen

mov ah, 4Ch       ; AH = 4Ch selects DOS terminate program function

int 21h           ; Return control back to DOS

end main          ; Marks end of program
                  ; Tells assembler that execution starts at main
```

Interrupts:

https://yassinebridi.github.io/asm-docs/8086_bios_and_dos_interrupts.html